Fault-Tolerant Quantum Portfolio Optimizer

Step-by-step code walkthrough with figures generated from the project's own code

Figures and numbers in this guide are produced by running the code in the zip (illustrative 12-qubit instance from a synthetic Oct 2008). Data used for all performance numbers: SYNTHETIC DATA. They illustrate how the pipeline behaves; they do not describe real markets.

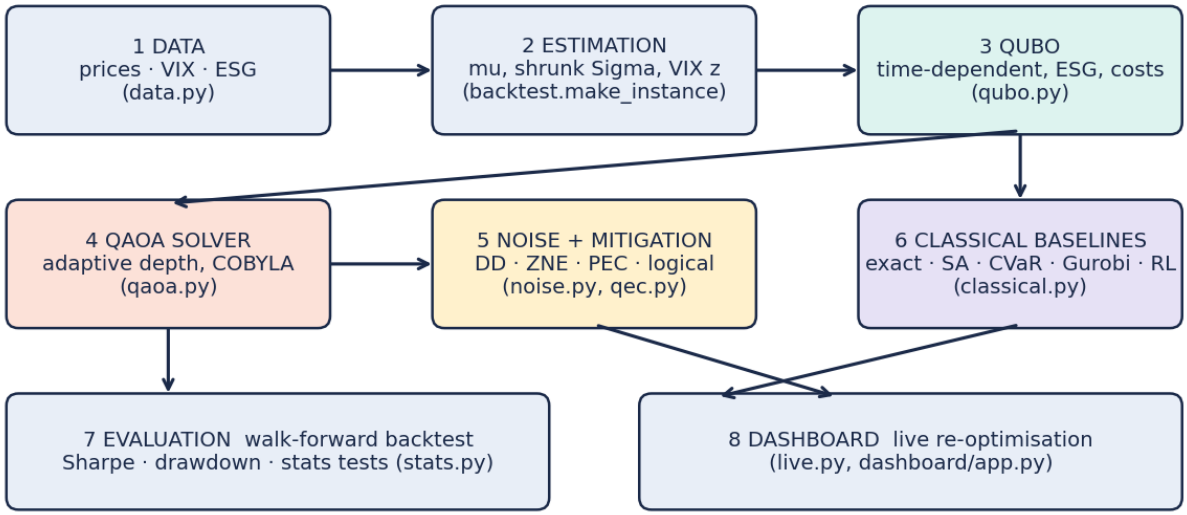
Contents

1. The big picture
2. Step 1 - Data
3. Step 2 - Estimate return and risk
4. Step 3 - Build the QUBO
5. Step 4 - QAOA (the quantum part)
6. Step 5 - Noise (NISQ+ model)
7. Step 6a - Zero-noise extrapolation (ZNE)
8. Step 6b - Probabilistic error cancellation (PEC)
9. Step 7 - Logical qubits with Stim
10. Step 8 - Resource estimation
11. Step 9 - Error-mitigation ablation
12. Step 10 - Classical baselines
13. Step 11 - Walk-forward backtest
14. Step 12 - Statistical significance
15. Step 13 - Scaling and theory bounds
16. Step 14 - Live dashboard

The big picture

Where: How the whole code runs, end to end

- Data comes in: prices, VIX, ESG scores.
- Every month we estimate returns (mu) and risk (Sigma).
- We turn 'choose the best weights' into a QUBO: a yes/no puzzle over bits.
- QAOA (a quantum circuit) tries to find the lowest-energy bit pattern.
- Noise is added, then error mitigation or logical qubits are applied.
- Classical solvers (exact, SA, MIQP, CVaR, RL) solve the same problem; a walk-forward backtest compares all of them.



run_all.py calls each box in this order, one command per stage.

Takeaway: Each box is one file in qpo/. Numbers in this deck: SYNTHETIC DATA.

Step 1 - Data

Where: qpo/data.py, scripts/fetch_data.py, scripts/build_esg.py

- Real mode: fetch_data.py freezes Yahoo prices and FRED series to csv; every later stage runs offline from that snapshot.
- ESG: build_esg.py converts Sustainalytics risk scores (lower = better) into a percentile score where higher = better.
- Safeguards: macro series are never back-filled, truncated FRED series are dropped, pseudo-ESG is refused.
- Offline mode: a synthetic market with calm and stress regimes and forced 2008-like and 2020-like crises (figure).
- This deck was built from: SYNTHETIC DATA (meta.json data_source = synthetic).

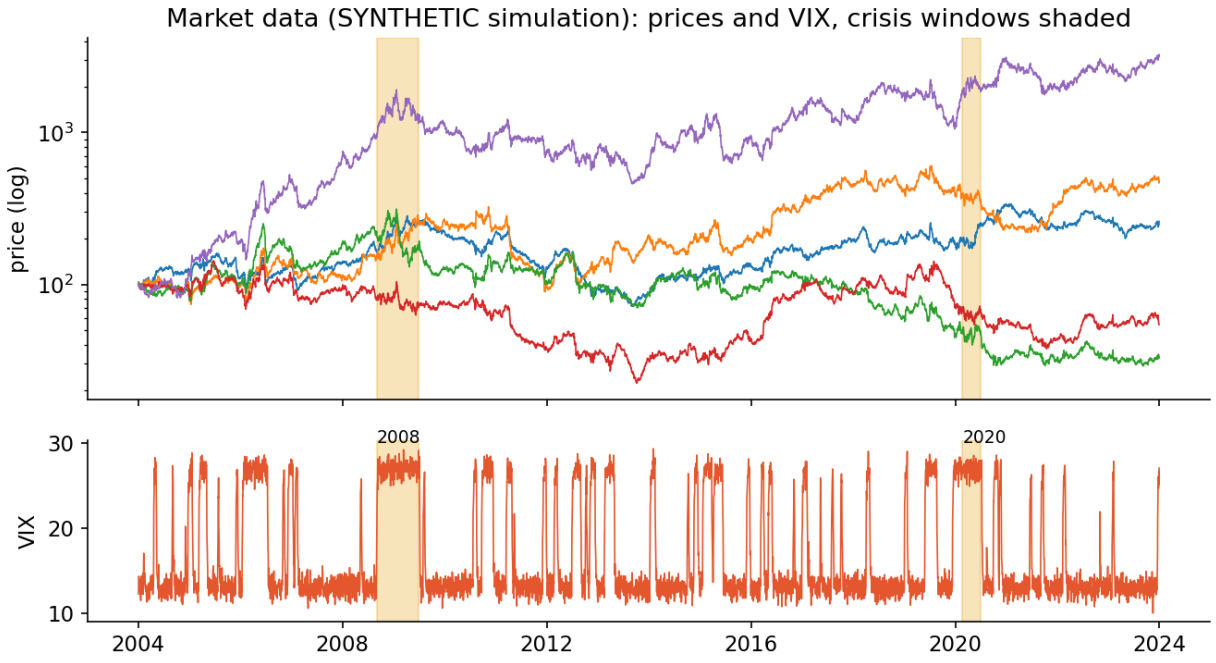


Figure is the synthetic market used for tests and demos.

Takeaway: Real runs must report survivorship bias and static-ESG look-ahead.

Step 2 - Estimate return and risk

Where: qpo/backtest.py make_instance()

- Use the last 252 trading days only (no look-ahead).
- Expected return mu = half the asset's own mean + half the universe mean (shrinkage).
- Covariance Sigma is shrunk 20% toward its diagonal.
- ESG hard filter: assets with ESG < 40 are removed first.
- Greedy pick of 3-4 assets (best Sharpe, low correlation) to fit on qubits.
- VIX z-score sets risk aversion lambda_t and the weight cap (50% calm, 33% stress).

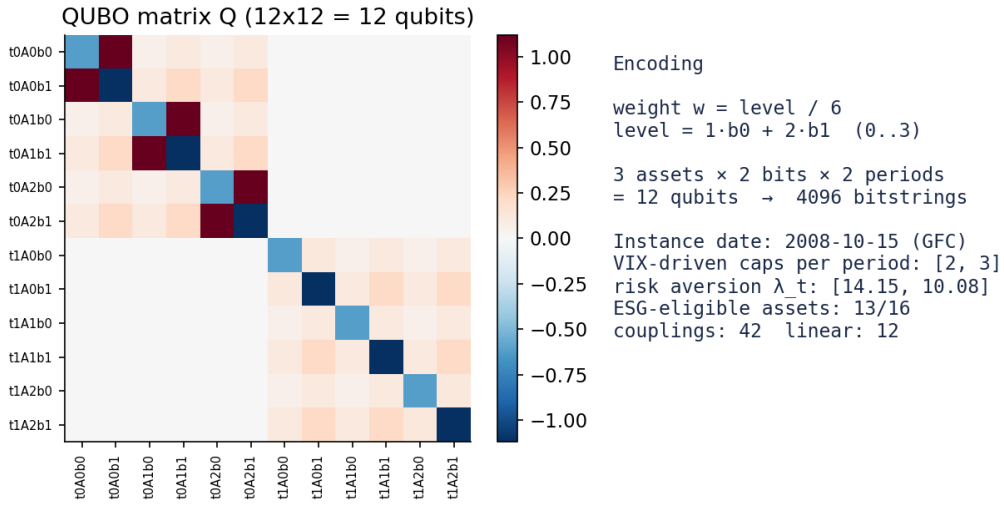
lambda_t = 6 x (1 + 0.5 x VIX_z). Stress: cap drops from 3/6 to 2/6.

Takeaway: Time-varying constraints come from this one step.

Step 3 - Build the QUBO

Where: qpo/qubo.py build_qubo()

- A QUBO is: minimise x^T Q x with every x a bit.
- Each weight = level/6, level = b0 + 2*b1: two bits per asset per period.
- 3 assets x 2 bits x 2 periods = 12 qubits (4096 candidate answers).
- Q mixes: -return, +risk, +transaction cost, +budget penalty, +cap penalty, -ESG tilt.
- Tests: Q matches the direct objective, and its optimum equals the hard-budget MIQP optimum.



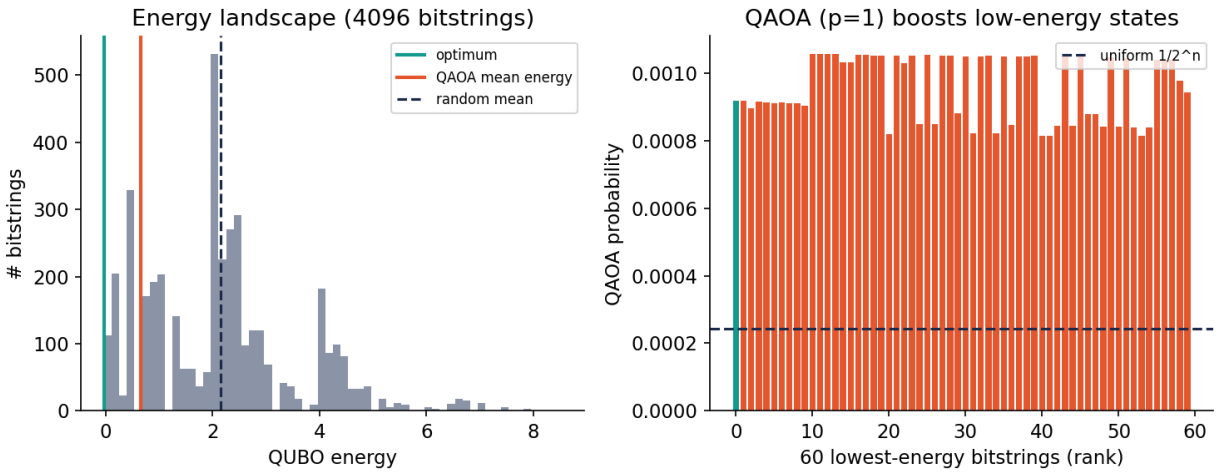
Heatmap of Q for the 2008-10-15 instance (synthetic market).

Takeaway: Red/blue cells are the couplings the circuit must encode.

Step 4 - QAOA (the quantum part)

Where: qpo/qaoa.py

- Start in an equal superposition of all bitstrings.
- Repeat p times: a cost phase then an RX mixer.
- Two angles per layer are tuned by COBYLA.
- Adaptive depth: add layers (up to 3) only while the improvement exceeds 1%.
- Sample bitstrings, evaluate their true energy classically, keep the best.



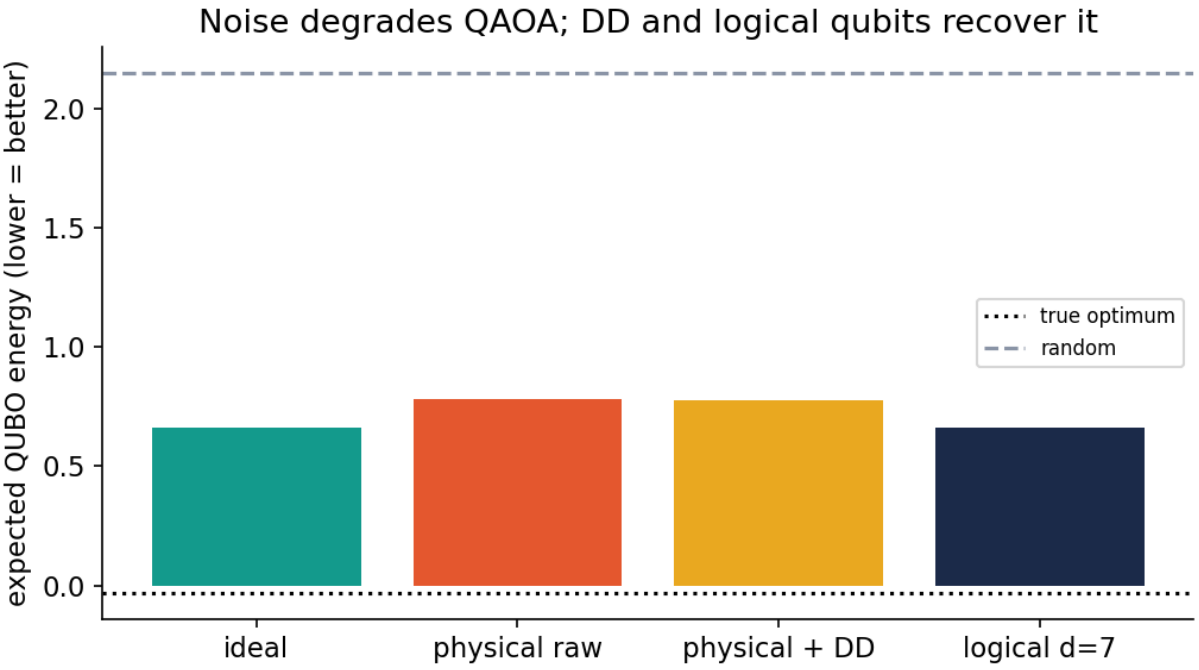
Optimum energy -0.031, random mean 2.149, QAOA (p=1) mean 0.663.

Takeaway: P(optimum) = 0.0009 vs uniform 0.00024: boosted, but the optimum stays unlikely.

Step 5 - Noise (NISQ+ model)

Where: qpo/noise.py

- Gate, idle and readout errors are turned into an independent bit-flip probability q per qubit on the output string.
- On this instance q is 0.026 (raw), 0.025 (with DD) and 1.0e-06 (logical).
- Phenomenological model, not hardware truth.



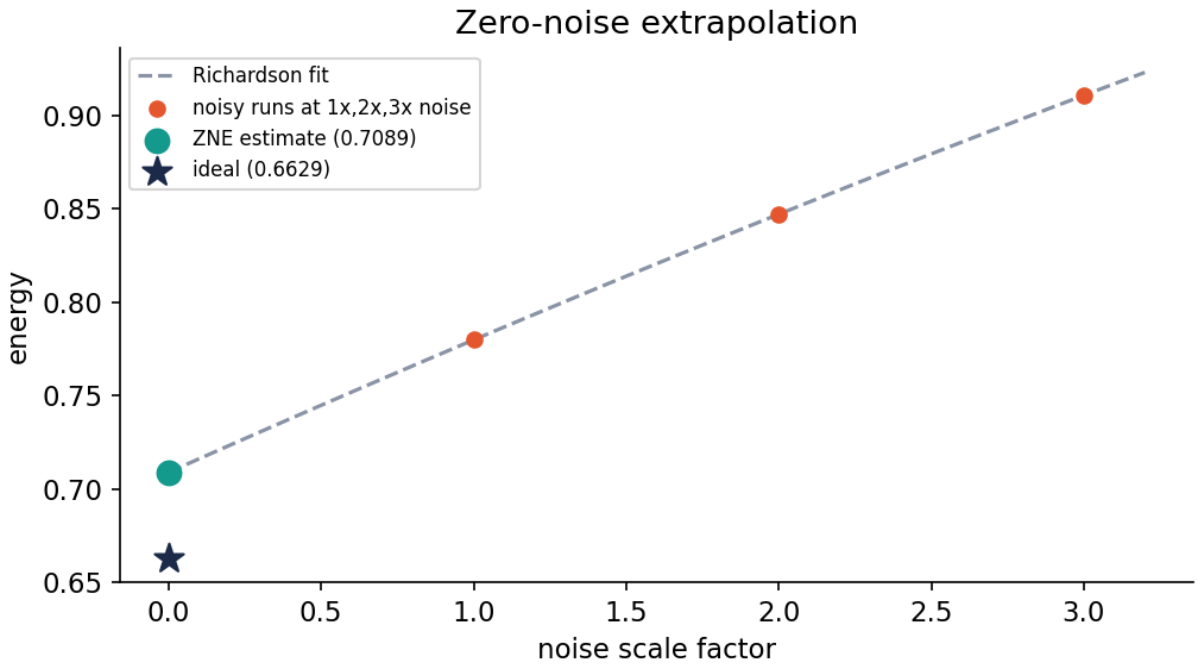
Energy 0.663 (ideal) -> 0.780 (raw).

Takeaway: Dynamical decoupling only trims idle error, so it helps a little.

Step 6a - Zero-noise extrapolation (ZNE)

Where: qpo/qaoa.py estimate_energy(method='zne')

- Run at 1x, 2x and 3x noise.
- Fit a curve and read it at 0x.
- Cost: more runs and variance inflated by 19x (Richardson weights 3, -3, 1).



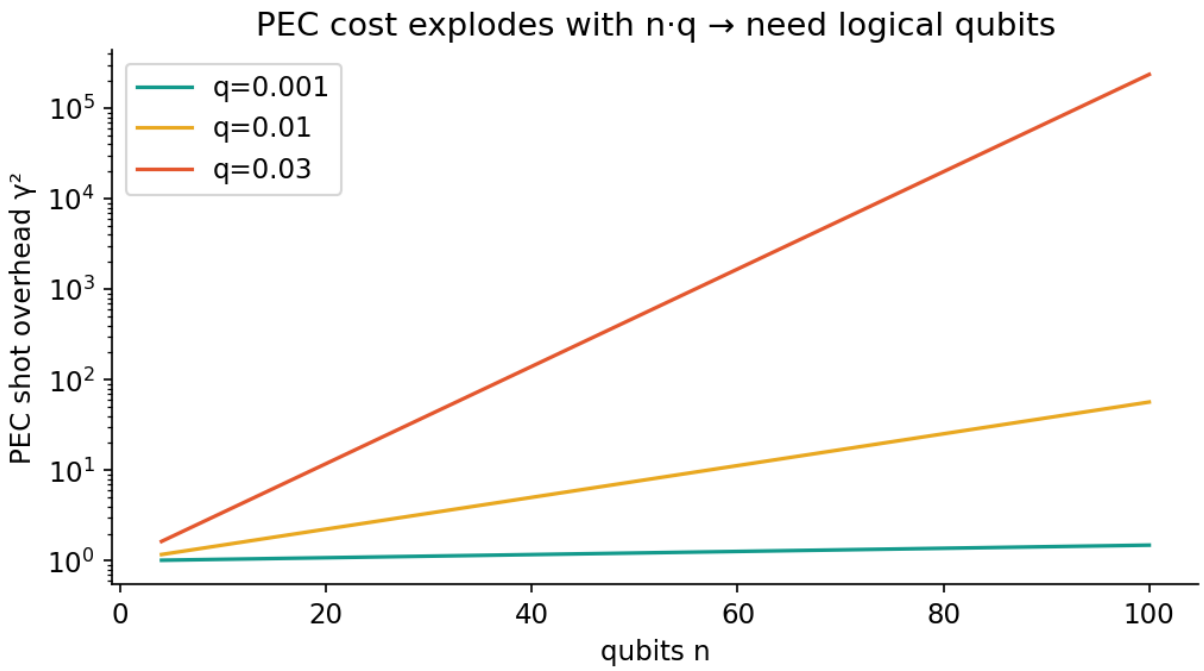
Raw 0.7798 -> ZNE 0.7089 (ideal 0.6629).

Takeaway: ZNE reduces the bias but not completely.

Step 6b - Probabilistic error cancellation (PEC)

Where: qpo/qaoa.py + qpo/theory.py

- The channel is known, so we apply its inverse with signed random samples.
- On this instance PEC gives 0.6629 (ideal 0.6629): unbiased.
- Price: gamma = (1-2q)^(-n); shots grow as gamma^2.



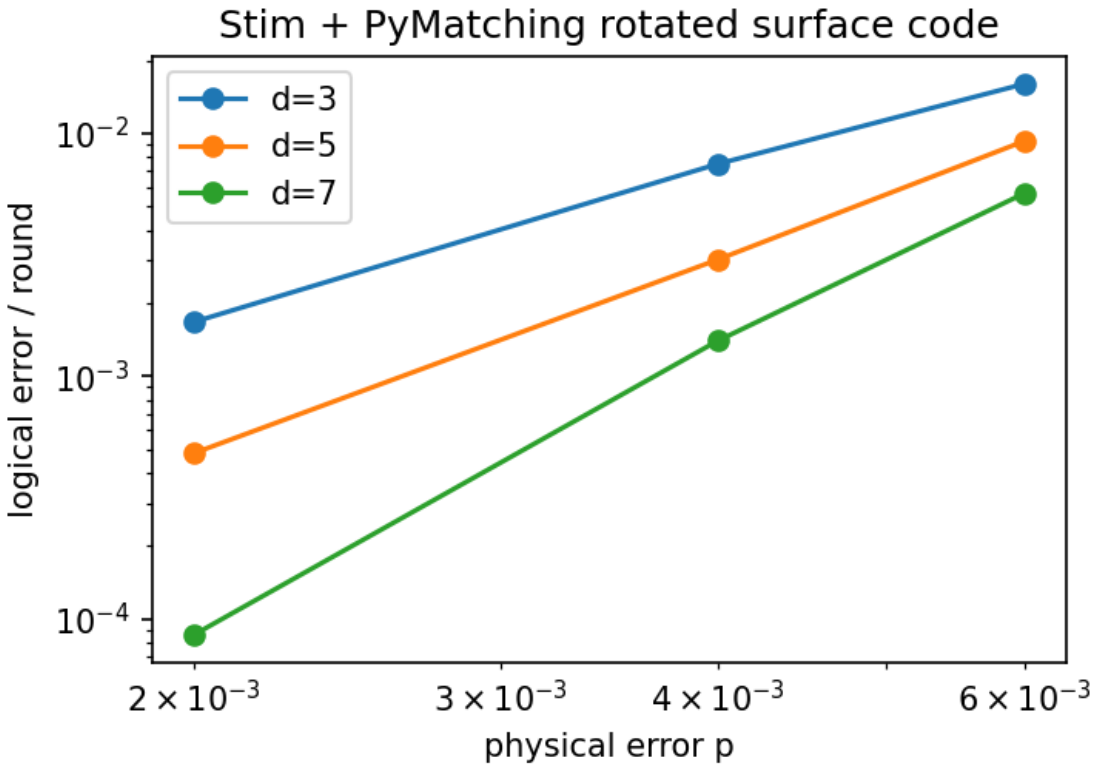
n=12 at this q: gamma^2 = 3.6. n=100, q=0.03: gamma^2 = 2.37e+05.

Takeaway: This is why logical qubits are needed at scale.

Step 7 - Logical qubits with Stim

Where: qpo/qec.py

- Stim simulates the rotated surface code memory experiment, PyMatching decodes (d = 3, 5, 7).
- Fit: p_L/round = A (p/p_th)^((d+1)/2) with A = 0.044, p_th = 0.0099 (9 points).
- Logical mode injects the fitted logical flip rate into the small QAOA simulation.
- Stim covers memory only; QAOA's non-Clifford gates are not simulated at code level.



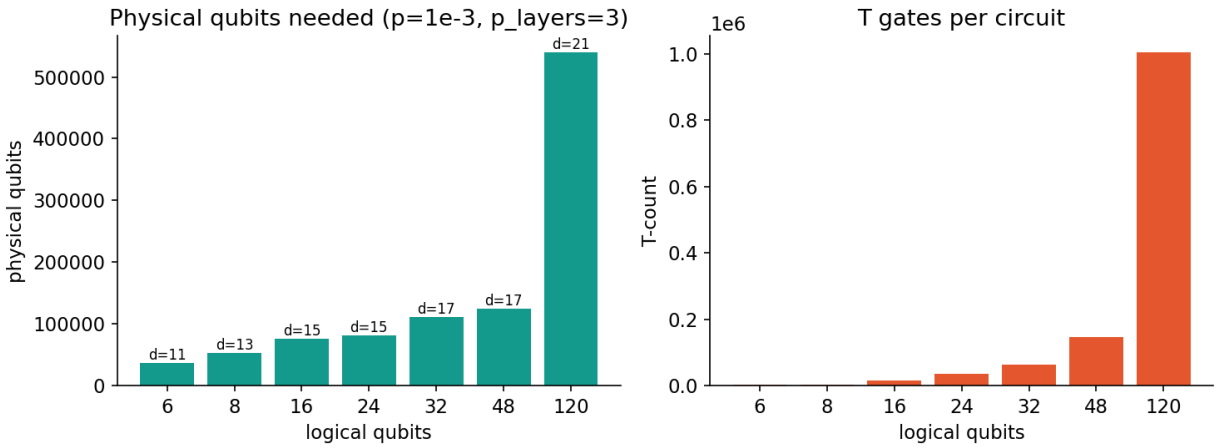
Data exist for p = 2e-3..6e-3 only; lower p is an extrapolation.

Takeaway: Say this clearly in the report.

Step 8 - Resource estimation

Where: qpo/qec.py estimate_resources(), scripts/resource_sweep.py

- Count rotations -> T gates, choose distance d, add T-factory area; failure budget split three ways (logical, synthesis, magic states).
- n=16, p=1e-3: 75,576 physical qubits (range 49,232 - 88,758).
- n=120, p=1e-3: 539,604 (range 183,480 - 1,180,620); needs 2-level distillation.
- All constants are swept (assumptions table, tornado plot). Runtime is T-supply bound.



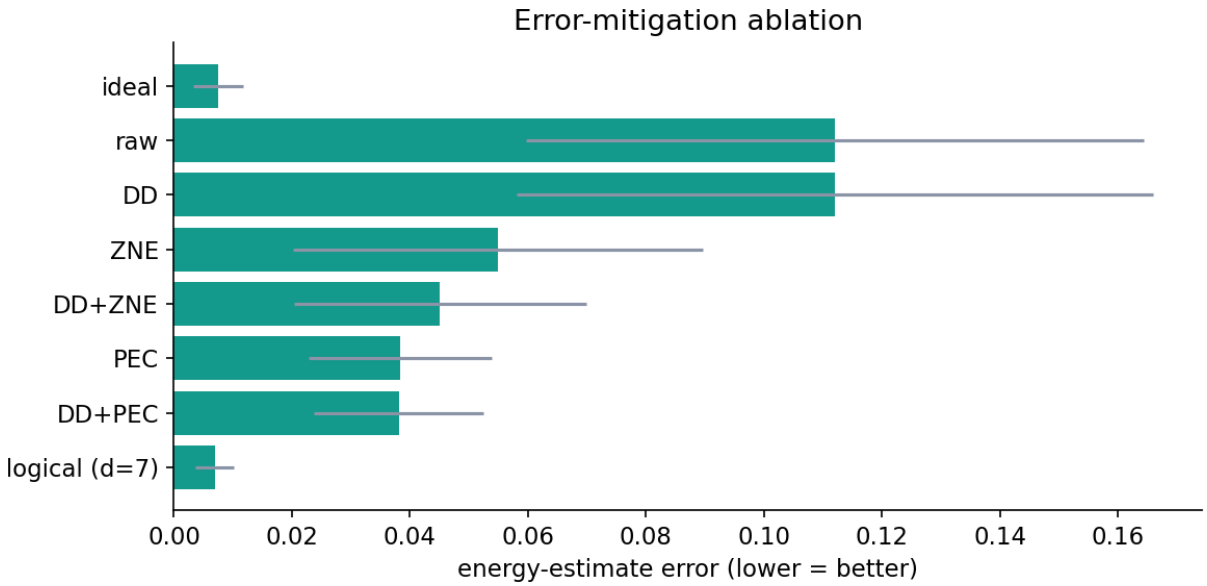
Report item (a).

Takeaway: Better hardware (p=1e-4) cuts the qubit count several-fold.

Step 9 - Error-mitigation ablation

Where: scripts/ablation.py

- Each mitigation is switched on separately and in combination (9 configurations).
- Raw error 0.112; ZNE 0.055; PEC 0.038.
- DD+ZNE 0.045; DD+PEC 0.038; logical (d=7) 0.007.
- Sampling overhead: ZNE 57x, PEC 12x.



Report item (b).

Takeaway: Mitigation helps estimation; logical qubits help most.

Step 10 - Classical baselines

Where: qpo/classical.py, qpo/miqp.py, qpo/rl_torch.py

- exact_qubo: brute force = true QUBO optimum (ceiling for QUBO solvers). sa_qubo: simulated annealing.
- miqp_sub: hard-budget discrete problem, Gurobi MIQP if installed, else an exact DP (same optimum, cross-tested).
- CVaR stochastic programs in HiGHS (and Gurobi when installed); equal weight.
- RL: NumPy evolution-strategy agent; PyTorch REINFORCE agent written but NOT run in this build.

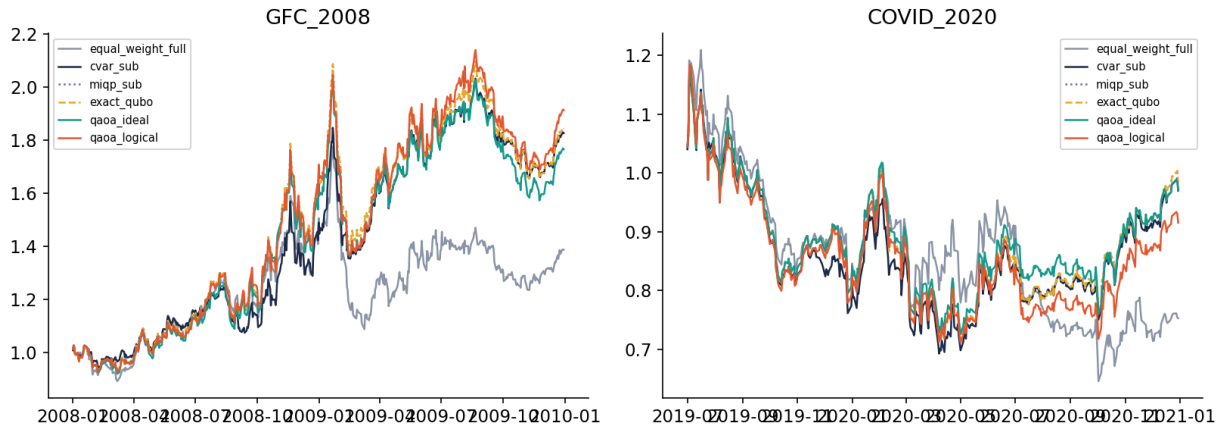
CVaR minimises average loss in the worst 10% of scenarios.

Takeaway: The exact optimum lets us measure how far QAOA is from perfect.

Step 11 - Walk-forward backtest

Where: qpo/backtest.py run_backtest()

- Every rebalance: build the QUBO from past data only, solve, trade, pay costs, hold.
- Methods: 11.
- GFC_2008: best classical cvar_sub Sharpe 1.17; QAOA logical 1.16; equal weight 0.67.
- COVID_2020: best classical cvar_full Sharpe 0.37; QAOA logical -0.01; equal weight -0.34.
- QUBO solvers differ little: the problem is tiny and the exact optimum is the ceiling.



SYNTHETIC DATA. These numbers say nothing about real markets.

Takeaway: Tests confirm there is no look-ahead.

Step 12 - Statistical significance

Where: qpo/stats.py

- Paired stationary bootstrap on the Sharpe difference; Wilcoxon on per-period returns; Holm correction.
- Result: 0 of 28 (GFC) and 0 of 28 (COVID) differences are significant.
- Example: qaoa_ideal vs cvar_sub in 2008: diff -0.13, 95% CI [-0.48, +0.32].
- Wide intervals mean low power: not significant is not the same as equal.

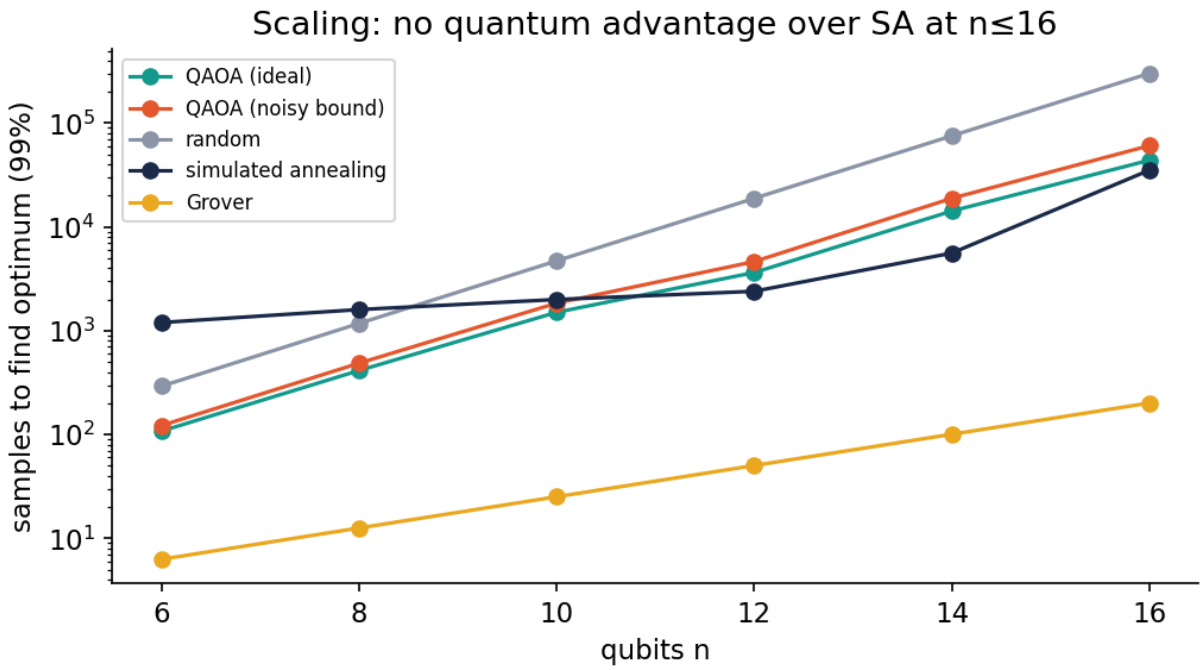
Report item (c).

Takeaway: No claim of quantum outperformance is supported by this table.

Step 13 - Scaling and theory bounds

Where: qpo/theory.py, docs/THEORY.md, scripts/scaling.py

- P1 noisy success >= ideal x (1-q)^n. P2 Hoeffding shots. P3 PEC cost gamma^2 (n=100, q=0.03: 2.4e5). P4 ZNE variance x19. P5 random baseline 2^n. P6 noise budget shrinks like 1/n.
- Empirical exponents (cost ~ 2^(s n)): QAOA 0.86, SA 0.43, random 1.00, Grover 0.50.
- Six sizes, n <= 16: trend evidence only.



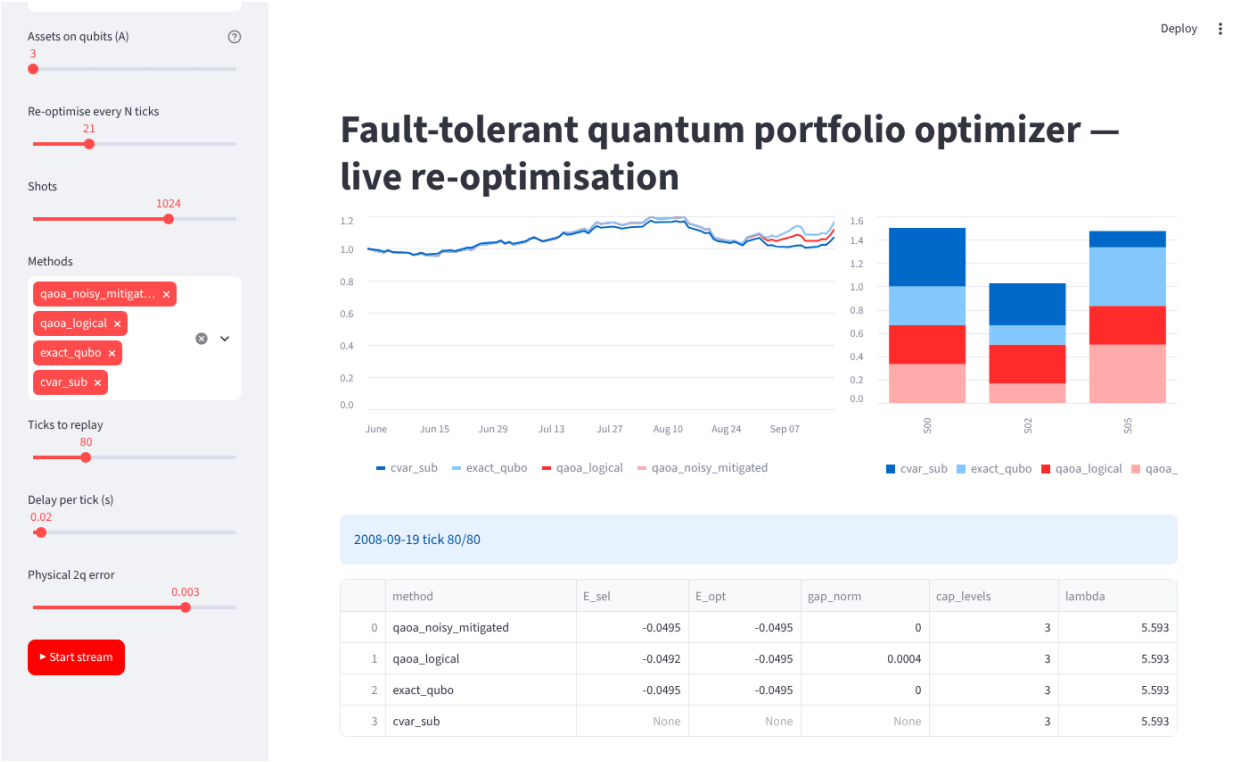
Honest result: the bounds are rigorous; an advantage is not shown.

Takeaway: Grover's sqrt(2^n) curve is a reference line only.

Step 14 - Live dashboard

Where: dashboard/app.py, qpo/live.py, scripts/screenshots.py

- Streamlit app replays market data tick by tick (synthetic or the real snapshot, with a banner saying which).
- Every N ticks it rebuilds the QUBO from data seen so far and re-solves with each chosen method.
- Shows P&L; lines, weight bars and a table of solver gaps.
- Screenshots below are from an EARLIER build on synthetic data; regenerate with scripts/screenshots.py.



	method	E_sel	E_opt	gap_norm	cap_levels	lambda
0	qaoa_noisy_mitigated	-0.0495	-0.0495	0	3	5.593
1	qaoa_logical	-0.0492	-0.0495	0.0004	3	5.593
2	exact_qubo	-0.0495	-0.0495	0	3	5.593
3	cvar_sub	None	None	None	3	5.593

Screenshot: 2008 replay, synthetic data, earlier build.

Takeaway: Report item (d).